

Análisis del algoritmo de gradiente descendente y estudio empírico comparativo con técnicas metaheurísticas

Autor: Eduardo Morales Muñoz

Doble Grado Ingeniería Informática y Matemáticas

Tutores: Javier Meri de la Maza
Pablo Mesejo Santiago

Junio 2025



Contenido

1. Introducción y contexto

2. Parte matemática

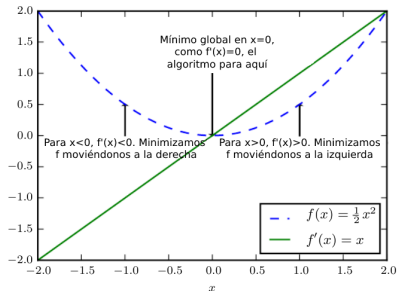
- Introducción
- *Backpropagation*
- Convergencia
- Conclusiones

3. Parte informática

- Introducción
- Estado del Arte
- Métodos y materiales
- Experimentación y resultados
- Conclusiones

Introducción

- Contexto: **aprendizaje profundo supervisado**.
- Entrenamiento: **problema de optimización**, ajustar los pesos para minimizar el error del modelo.
- Entrenamiento = **gradiente descendente (GD)**.



GD: ajusta iterativamente los parámetros de un modelo en la dirección opuesta al gradiente.

Gradiente Descendente

Ventajas

- **Escalable** a grandes conjuntos de datos.
- **Eficiente** en términos computacionales.
- **Flexible** ya que se adapta a distintos modelos y tareas.
- Bien estudiado y respaldado teóricamente.

Limitaciones

- Puede converger a **mínimos locales subóptimos**.
- Se estanca en **puntos de silla**.
- Sufre de **explosión o desaparición del gradiente**, sobre todo en redes profundas.
- Sensible a hiperparámetros.

Motivación

- **Los costes del entrenamiento pueden ser inasequibles** en términos computacionales y económicos.
- **Una mejora** de GD tendría un **impacto directo en la eficiencia** del entrenamiento de redes profundas.
- Podemos **explorar nuevas estrategias** para superar los límites del GD.

Parte matemática

Estudio teórico del gradiente descendente

Introducción y Objetivos

$$W_{t+1} = W_t - \eta \nabla C(W)$$

Introducción y Objetivos

$$W_{t+1} = W_t - \eta \nabla C(W)$$

- ¿Cómo calculamos el gradiente? → **Backpropagation.**

Introducción y Objetivos

$$W_{t+1} = W_t - \eta \nabla C(W)$$

- ¿Cómo calculamos el gradiente? → **Backpropagation**.
- ¿Vamos a obtener una buena solución? → **Convergencia**.

Introducción y Objetivos

$$W_{t+1} = W_t - \eta \nabla C(W)$$

- ¿Cómo calculamos el gradiente? → **Backpropagation**.
- ¿Vamos a obtener una buena solución? → **Convergencia**.
- **Objetivos:**
 - Estudio del algoritmo de **Backpropagation**.
 - Análisis de la convergencia de GD tanto **Batch Gradient Descent** (BGD) como **Stochastic Gradient Descent** (SGD).

Comparativa: BGD vs. SGD

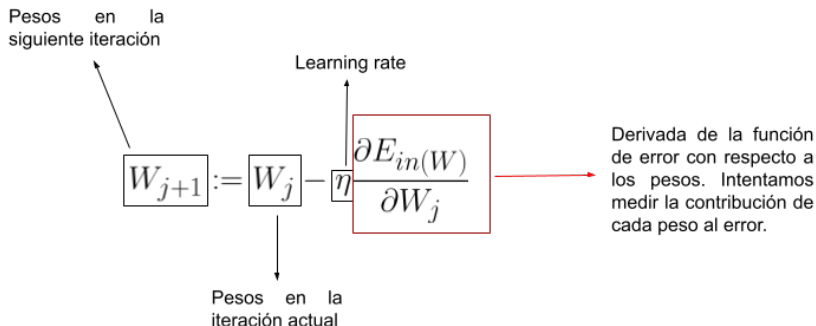
BGD

- **Determinista:** usa todo el conjunto de datos para calcular el gradiente.
- **Trayectoria suave** y estable del error.
- **Computacionalmente inasequible**, sólo es viable en conjuntos de datos pequeños.
- **Útil en contextos teóricos.**

SGD

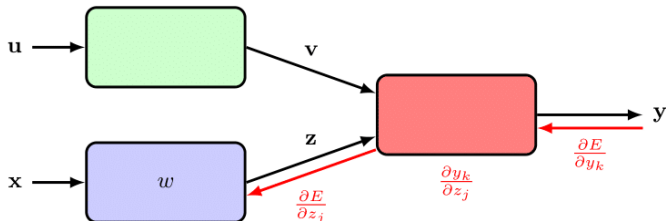
- **Estocástico:** actualiza con una muestra aleatoria o *minibatch*.
- **Mayor variabilidad** en el descenso del error.
- **Eficiente y escalable:** ideal para *big data* y aprendizaje profundo.
- **Desarrollos teóricos más complejos** debido a la componente estocástica.

Backpropagation I



- Es **prácticamente el único algoritmo** utilizado para el cálculo del gradiente.
- Es una **herramienta muy potente** que se basa en un concepto muy simple como es la regla de la cadena.

Backpropagation II



- **Forward pass:** se calcula la salida del modelo y el error cometido, guardando cálculos intermedios que pueden servir en la siguiente fase.
- **Backward pass:** se propaga el error del modelo hacia atrás para obtener los gradientes capa a capa.

Convergencia BGD

Convergencia para funciones convexas

Sea $f : \mathbb{R}^n \rightarrow \mathbb{R}$ nuestra función de coste, de clase $\mathcal{C}^2$, convexa, y con gradiente Lipschitz continuo con constante $L > 0$.

Entonces, si se aplica el algoritmo de BGD durante k iteraciones con un *learning rate* constante $\eta < \frac{1}{L}$, el error disminuye en cada iteración y se obtiene una solución $x^{(k)}$ que satisface:

$$f(x^{(k)}) - f(x^*) \leq \frac{\|x^{(0)} - x^*\|_2^2}{2\eta k},$$

donde x^* es el **mínimo global** de f .

Convergencia SGD I

Definición (Casi Supermartingala)

Sea $\{M_n\}_{n \geq 0}$ una sucesión de variables aleatorias tal que $\mathbb{E}[|M_n|] < \infty$. Decimos que es una **casi supermartingala** con respecto a $\{\mathcal{F}_n\}_{n \geq 0}$ si para todo $m < n$:

$$\mathbb{E}[M_n \mid \mathcal{F}_m] \leq (1 + \beta_m)M_m + \xi_m - \zeta_m,$$

donde $\beta_m, \xi_m, \zeta_m \geq 0$ y son $\mathcal{F}_m$ -medibles.

- Esto implica que M_n **no puede crecer mucho en promedio**, aunque puede decrecer.
- Si $M_n \geq 0 \quad \forall n \in \mathbb{N}$, decimos que es no negativa.
- **Modelamos el proceso de SGD como una casi supermartingala no negativa.**

Convergencia SGD II

Convergencia para SGD

Sea:

$$V_n = \|W_n - W^*\|^2,$$

donde W_n son los pesos en la iteración n y W^* son los pesos óptimos, se tiene:

- Si $\sum_{n=1}^{\infty} \eta_n^2 < \infty$ casi seguramente, entonces $V_n \rightarrow V_{\infty}$ c.s.
- Si además $\sum_{n=1}^{\infty} \eta_n = \infty$ casi seguramente, entonces:

$$V_{\infty} = 0 \quad \text{y} \quad \lim_{n \rightarrow \infty} W_n = W^* \quad \text{casi seguramente.}$$

- La función de coste debe ser **globalmente fuertemente convexa**.

Conclusiones matemáticas

- **BGD:**
 - El error **disminuye en cada iteración**.
 - **Convergencia monótona al óptimo** en $\mathcal{O}(1/k)$.

Conclusiones matemáticas

■ BGD:

- El error **disminuye en cada iteración**.
- **Convergencia monótona al óptimo** en $\mathcal{O}(1/k)$.

■ SGD:

- **Teoría de martingalas** como herramienta para estudiar la convergencia.
- **Convergencia casi segura al óptimo** ajustando el *learning rate*.
- Aunque la hipótesis de la convexidad fuerte y global sea demasiado estricta, **se espera que los resultados se mantengan a nivel local**.

Parte informática

Estudio comparativo del gradiente descendente con técnicas
metaheurísticas

Buscar otras estrategias

- El análisis teórico tiene inconvenientes:
 - **Hipótesis demasiado estrictas** para la función de coste.
 - Deberíamos realizar un **análisis específico de cada problema**.
 - Dicho análisis puede requerir **recursos computacionales considerables**.
- Buscamos **estrategias alternativas** que no necesiten que la función de coste sea convexa.

Metaheurísticas como alternativa

- **Metaheurísticas (MH):** flexibles y adaptables. Soluciones aproximadas para problemas complejos.
- **Ventajas vs GD:**
 - No requieren convexidad.
 - Exploración global.
 - Mayor robustez con superficies de error complejas.
 - Fácilmente adaptables a diferentes dominios.
- **Limitaciones:** peor rendimiento actualmente que el GD, especialmente en problemas grandes.

Objetivos

- **Evaluar y analizar la eficacia de las técnicas MH** para el entrenamiento de redes profundas en comparación con el **GD**.
 - **Diferencia de rendimiento** según el tipo de tarea.
 - **Factores determinantes** del mismo.
 - **Tiempos de ejecución**.
- **Propuesta de algoritmos híbridos: SHADE-GD y SHADE-ILS-GD**

Estado del arte

Entrenamiento de modelos:

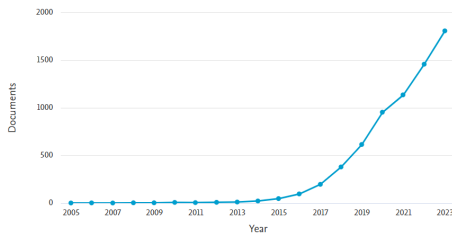
■ MH puras:

- Algoritmos evolutivos, enjambre de partículas, colonia de hormigas, etc.

■ MH + GD:

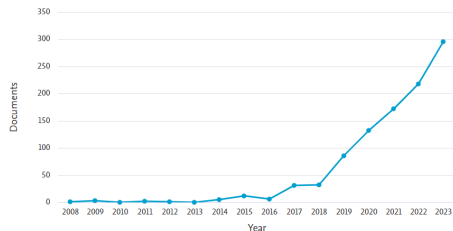
- MH como inicializadores o guías para GD.
- Híbridos iterativos o en paralelo: exploran zonas globales y luego ajustan localmente con GD.

Documents by year



GD

Documents by year



MH

Desarrollo

- Amplia batería experimental.
- Diversos modelos (MLPs y CNNs) entrenados *from scratch*.
- Implementamos MH a partir de una librería base.
- Adaptamos librerías de aprendizaje profundo a las MH.
- El código puede encontrarse en: <https://github.com/eedduu/TFG>.

Métricas de rendimiento

- **Accuracy (Precisión global):** proporción de predicciones correctas.

$$\text{Accuracy} = \frac{\text{Nº aciertos}}{\text{Total de muestras}}$$

- **Balanced Accuracy (Precisión equilibrada):** *datasets* desbalanceados. Penaliza ignorar clases minoritarias.

$$\text{Balanced Accuracy} = \frac{1}{C} \sum_{i=1}^C \frac{\text{TP}_i}{\text{TP}_i + \text{FN}_i}$$

- **Coeficiente de determinación** R^2 : proporción de varianza explicada.

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Fundamentos teóricos

■ **Análisis de Dependencias Parciales (PDP):**

- Interpretación de modelos.
- Efecto de cada variable sobre la predicción.
- Visualizamos relaciones no lineales.

■ **Test de normalidad de Shapiro-Wilk:**

- Muestras pequeñas o medianas.
- Hipótesis nula: los datos siguen una distribución normal.

■ **Test de los rangos con signo de Wilcoxon:**

- Alternativa al test t sin normalidad.
- Hipótesis nula: las diferencias tienen mediana cero.

I Rendimiento de MLPs según el tipo de tarea

- **Hipótesis:** Los MLP entrenados con MH presentan diferencias de rendimiento significativas respecto al GD en regresión pero no en clasificación.
- **Experimento:**
 - Entrenar MLPs en conjuntos de datos tabulares con características similares.
 - Comparar optimizadores basados en GD frente a MH.
- **Métrica :**

$$\text{Dif}_{\text{rel}} = \frac{\text{Metric}_{\text{MH}} - \text{Metric}_{\text{GD}}}{|\text{Metric}_{\text{GD}}|}$$

- $\text{Dif}_{\text{rel}} = 0$: rendimiento igual.
- $\text{Dif}_{\text{rel}} < 0$: GD supera a MH.
- $\text{Dif}_{\text{rel}} > 0$: MH supera a GD.

I Rendimiento de MLPs según tipo de tarea

Tarea	Rango Dif_{rel}	Media
Clasificación	$-0,725$ a $+0,010$	$-0,35$
Regresión	$-5,98 \times 10^5$ a $-0,27$	$\ll 0$

Test	Resultado
Shapiro–Wilk (normalidad)	$p = 0,003$
Wilcoxon rangos con signo	$p = 0,023$

Observaciones clave:

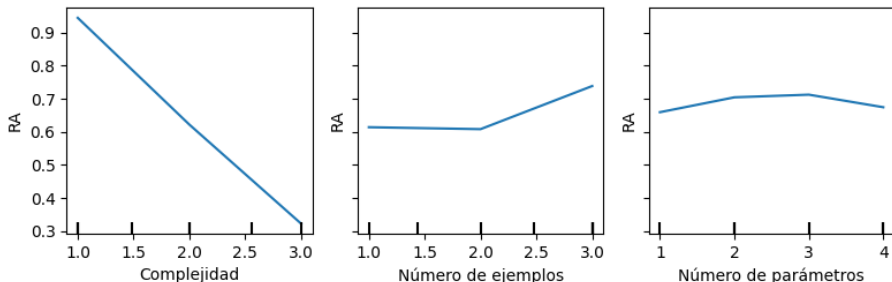
- Diferencias **estadísticamente significativas** según el tipo de tarea.
- Diferencias acentuadas **con mayor profundidad de la red**.

II Factores de pérdida de rendimiento en GD y MH en clasificación

- **Hipótesis:** La pérdida de rendimiento en clasificación de las MH está fuertemente influenciada por:
 - *Cantidad de ejemplos.*
 - *Número de parámetros del modelo.*
 - *Complejidad del conjunto de datos.*
- **Experimento:** medir el impacto de cada uno.
- **Métrica:** $RA = \frac{\text{accuracy} - \text{accuracy}_{\text{aleatorio}}}{\text{accuracy}_{\text{aleatorio}}}$
 - $RA = 0$: igual al clasificador aleatorio.
 - $RA < 0$: peor que aleatorio.
 - $RA > 0$: mejor que aleatorio.

II Factores de pérdida de rendimiento en GD y MH en clasificación

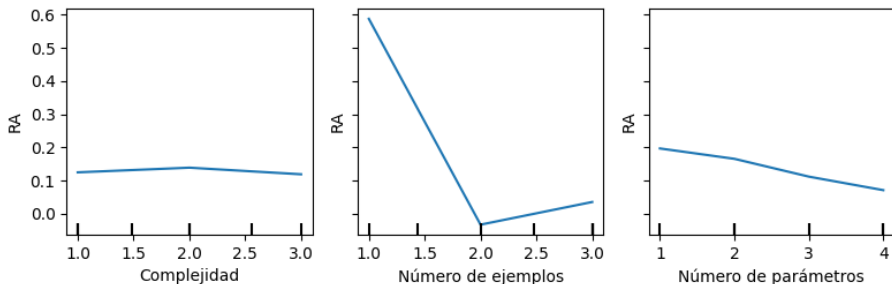
Análisis de Dependencias Parciales (GD)



- Complejidad: fuerte caída → **gran impacto negativo**.
- Parámetros y ejemplos: efectos suaves (mejor con más datos, cuidado con sobreajuste).

II Factores de pérdida de rendimiento en GD y MH en clasificación

Análisis de Dependencias Parciales (MH)



- Complejidad y parámetros: casi planos → **sin impacto**.
- Nº ejemplos: pendiente negativa pronunciada → **más datos** ⇒ **peor rendimiento**.

III Análisis de tiempos de ejecución

- **Hipótesis:** El tiempo de ejecución de las MH viene influenciado por:
 - Número de instancias del conjunto.
 - Número de parámetros del modelo.

- **Experimento:**
 - 1 Medir cuál tiene más peso.
 - 2 Desglosar el tiempo de ejecución de las MH.

III Análisis de tiempos de ejecución

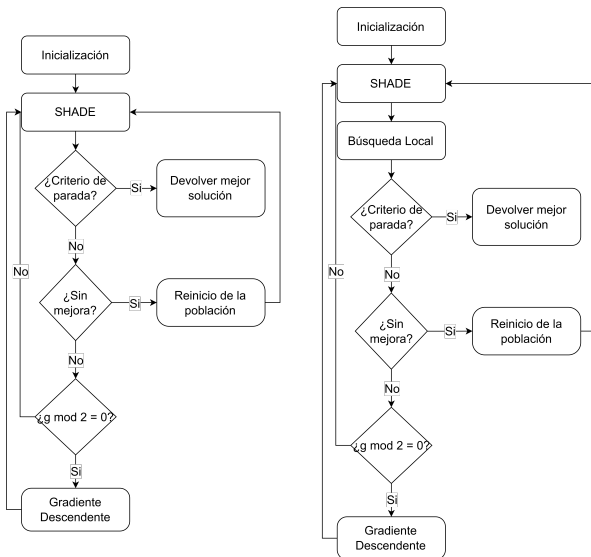
Factor	Impacto relativo
Tamaño de instancia vs. parámetros	$7\times$ a $10\times$ más impacto

Componente (MH)	Proporción de tiempo
Evaluaciones de entrenamiento	$\approx 77\%$
Mutación / Cruce	$\approx 15\%$
Búsqueda local	$\approx 7\%$
Guardado / Validación	$\approx 1\%$

Observaciones clave:

- El **número de instancias impacta mucho más** que el de parámetros.
- MH pueden ser **más rápidas por época**.
- **Tiempo total mucho mayor en MH** debido al número de épocas requeridas.

IV Técnicas propuestas



Objetivo:

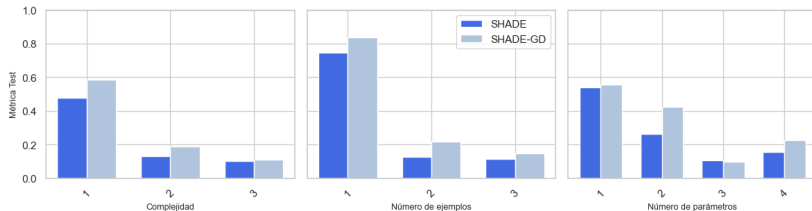
Combinar búsqueda global (MH) y local (GD) para entrenar modelos profundos.

Base:

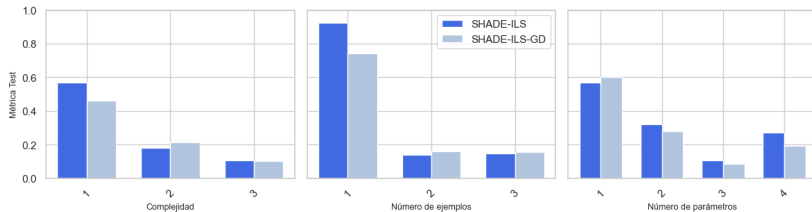
- **SHADE:** Evolución Diferencial auto-adaptativa.
- **SHADE-ILS:** SHADE con búsqueda local iterativa.

IV Técnicas propuestas

Comparativa SHADE vs SHADE-GD



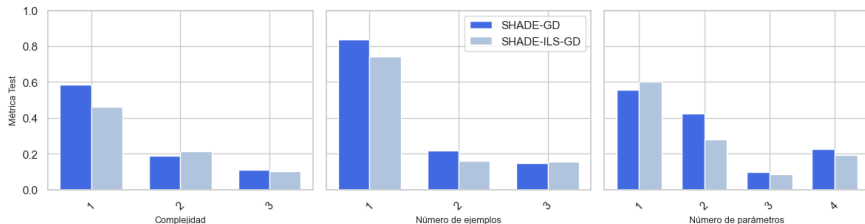
Comparativa SHADE-ILS vs SHADE-ILS-GD



IV Técnicas propuestas (SHADE-GD vs SHADE-ILS-GD)

- SHADE-ILS-GD reduce más el error y destaca en redes simples, pero sobreajusta.
- SHADE-GD generaliza mejor, es más consistente y obtiene mejor rendimiento en la mayoría de tareas.

Comparativa SHADE-GD vs SHADE-ILS-GD



Conclusiones clave

Rendimiento relativo

- Clasificación vs. regresión: $p = 0,023$.
- MH más sensibles al **número de datos** que a complejidad o parámetros.

Eficiencia temporal

- MH por época: **más rápidas** que GD.
- Entrenamiento global: **más lento** debido a más épocas.

Híbridos propuestos

- **SHADE-GD**: mejora consistente sobre SHADE.
- **SHADE-ILS-GD**: mejoras puntuales, no competitivo en tareas complejas.

¡Gracias por su atención!